

Requirements Specification Document

Gather

CMSI 4071 - Senior Project I
Loyola Marymount University

Team Members

Daniel Ruskin
Aristotle Kaporis
Luca Piccinini
Ivana Krajina
Greg Forston

Fall 2026

Table of Contents

Requirements Specification Document.....	1
Gather.....	1
Table of Contents.....	1
Table of Figures.....	2
Table of Tables.....	3
5.1 Introduction.....	3
5.1.1 Identification.....	3
5.1.2 System Overview.....	3
5.2 Development Software and Resources.....	5
5.2.1 Development Apps.....	5
5.2.2 User Resources.....	5
5.3 Functional Requirements by CSC.....	6
5.3.1 GUI (Frontend React) Subsystem.....	6
5.3.2 Backend (Node JS + Express) Subsystem.....	9
5.4. Performance requirements by CSC:.....	11
5.5 Project Environment Requirements.....	12
5.5.1 Development Environment Requirements.....	13
5.5.2 Execution Environment Requirements.....	13

Table of Figures

Figure 1 - Gather Flowchart	3
Table 1 - Document Definitions	4
Table 2 - Development App Titles	4
Table 3 - User Resources	5
Table 4 - Development Environment Requirements	12
Table 5 - Execution Environment Requirements	13

5.1 Introduction

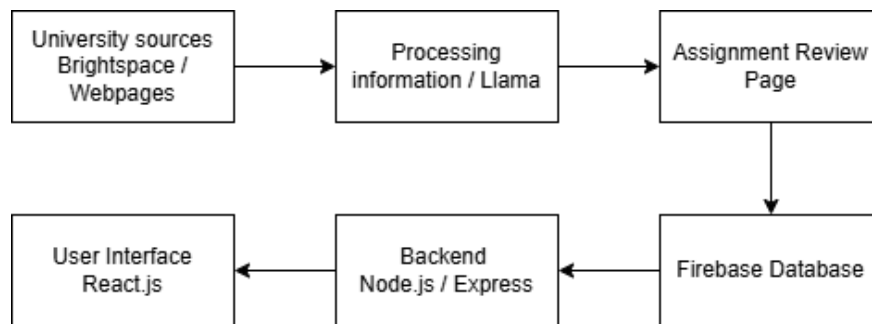
5.1.1 Identification

This document details the requirements for developing the task management Web Application Gather.

5.1.2 System Overview

The Gather system is designed as a platform that collects student-centralized information (due dates and events) across a variety of websites and documents with an intent to then display it as an organized “to-do” list. After identifying and collecting different due dates, assignments, exams, etc., the Gather system presents it to students asking for approval before they can be added to their task list.

The system consists of backend, data management and collection, user interface and artificial intelligence components that will be integrated across the Gather platform.



5.1.3 Document Overview

This document is organized as follows. Section 2 presents tables containing pertinent related documents that apply to this program. Section 3 contains a more in-depth description of the system operation. Section 4 provides details of computing, compiling, development, CM tools, and other supporting resources required. Section 5 presents the distribution and maintenance information. Section 6 contains a Verification Cross Reference Index (VCRI) table for tracking requirements verification tracking. Section 7 provides a Validation Cross Reference Matrix (VCRM) table, containing requirements verification information. Finally, section 8 contains appendices and assorted miscellaneous information.

The following table defines the language used in specifying requirements in this document. There are three levels of specification, only one of which constitutes and indicates an actual contractual, verifiable requirement.

Table 1- Document Definitions

Type	Definition
Shall	Expresses a mandatory provision. Any deviations from these contractually imposed mandatory requirements require the approval of the John Q. Public Software REA.
Should	Expresses non-mandatory provision. Unless required by other contract provisions, noncompliance with this requirement does not require John Q. Public Software REA approval and does not require documented technical substantiation.
Will	Declaration of purpose such as a design goal. Unless required by other contract provisions, noncompliance with this requirement does not require John Q. Public Software REA approval and does not require documented technical substantiation.

5.2 Development Software and Resources

5.2.1 Development Apps

The tables in the following subsections identify what software is going to be used by the development team in order to create the product.

Table 2- Development App Titles

Link	App Title	Reason
firebase.google.com/	Firebase	Database
react.dev/	React	Frontend Functionality
nodejs.org/	Node JS	JS Runtime Environment
expressjs.com/	Express JS	Backend for Node
openrouter.ai/	OpenRouter	API access to LLMs
docs.valence.desire2learn.com/reference.html	Brightspace API	API access to Brightspace

5.2.2 User Resources

Table 3- User Site Titles

Link	Site Title	Reason
brightspace.lmu.edu/d2l/home	Brightspace	Class task listing
...	Gather	The actual site
...	Professor Sites	Sites with information to add

5.3 Functional Requirements by CSC

5.3.1 GUI (Frontend React) Subsystem

The GUI subsystem provides the frontend interface for the user to view, review, and create assignments.

Authentication & Navigation

- **5.3.1.1** The GUI subsystem shall provide a Login Page for returning user authentication.
- **5.3.1.2** The GUI subsystem shall provide a Sign-Up Page for new user registration.
- **5.3.1.3** The GUI subsystem shall provide a Password Reset page or modal for unauthenticated users.
- **5.3.1.4** The GUI subsystem shall provide a Logout button accessible from the navigation bar or Settings Page.
- **5.3.1.5** The GUI subsystem shall include a navigation bar to allow switching between application pages.
- **5.3.1.6** The GUI subsystem shall provide a link in the navigation bar to access the Main Page.

- **5.3.1.7** The GUI subsystem shall provide a link in the navigation bar to access the Assignment Adding Page.
- **5.3.1.8** The GUI subsystem shall provide a link in the navigation bar to access the Settings Page.

Main Application Page

- **5.3.1.9** The Main Page shall display a main application window for the user.
- **5.3.1.10** The Main Page shall display a graphic indicating loading while loading.
- **5.3.1.11** The Main Page shall display a calendar view containing the user's assignments.
- **5.3.1.12** The Main Page shall display a complete list of all assignments, with due dates attached.
- **5.3.1.13** The Main Page shall allow users to filter the assignment list by active or completed status.
- **5.3.1.14** The Main Page shall automatically sort the assignment list chronologically by due date.
- **5.3.1.15** The Main Page shall provide a control (e.g., a checkbox or toggle) for the user to mark an assignment's completion status.
- **5.3.1.16** The Main Page shall provide options to edit or delete each assignment upon right-clicking it.
- **5.3.1.17** The Main Page shall display a pop-up with fields to edit an assignment's due date, title, and details when edit is selected, or delete the assignment when delete is selected.

Assignment Adding Page

- **5.3.1.17** The Assignment Adding Page shall contain fields for URLs, PDFs, and manual assignment creation, as well as a button for connecting to Brightspace.
- **5.3.1.18** The Assignment Adding Page shall redirect the user to the Assignment Review Page upon non-manual addition of assignments.
- **5.3.1.19** The Assignment Adding Page shall display a loading indicator while awaiting PDF, URL, or Brightspace data processing.
- **5.3.1.20** The GUI subsystem shall display a user-friendly error notification if PDF extraction, URL scraping, or Brightspace API fetching fails.

- **5.3.1.21** The Assignment Review Page shall display information on proposed assignments and provide controls to edit or delete them.
- **5.3.1.22** The Assignment Review Page shall provide a button to finalize changes and redirect the user to the Main Page.
- **5.3.1.23** The Assignment Adding Page shall display an assignment creation pop-up upon selecting manual assignment creation.
- **5.3.1.24** The assignment creation pop-up shall provide fields for assignment name, due date, and details.
- **5.3.1.25** The assignment creation pop-up shall provide a button to finalize the creation of the assignment.
- **5.3.1.26** The Assignment Adding Page shall redirect the user to Brightspace authentication and permissions prompts upon selecting to connect Brightspace.
- **5.3.1.27** The Assignment Adding Page shall display enrolled Brightspace classes after user permission is granted.
- **5.3.1.28** The Assignment Adding Page shall display a button to add a class when hovering over an enrolled class.
- **5.3.1.29** The GUI subsystem shall retrieve class assignments from the Brightspace API and present them on the Assignment Review Page upon adding a class.

Settings Page

- **5.3.1.30** The Settings Page shall provide controls for the user to adjust account settings.
- **5.3.1.31** The Settings Page shall display the user's account details.

5.3.2 Backend (Node JS + Express) Subsystem

The Backend subsystem serves as the intermediary for the GUI subsystem to interact with the Database subsystem.

Creating, Reading, Writing, Deleting Assignments

- **5.3.2.1** The Backend subsystem shall provide API endpoints to create assignments in the database.
- **5.3.2.2** The Backend subsystem shall provide API endpoints to delete assignments from the database.
- **5.3.2.3** The Backend subsystem shall provide API endpoints to edit assignments in the database.
- **5.3.2.4** The Backend subsystem shall provide API endpoints to read assignments from the database.

Brightspace API

- **5.3.2.5** The Backend subsystem shall authenticate with the Brightspace API and securely store access tokens upon successful authentication.
- **5.3.2.6** The Backend subsystem shall enforce secure communication channels when interacting with the Brightspace API.

- **5.3.2.7** The Backend subsystem shall provide API endpoints to read user enrolled classes from Brightspace.
- **5.3.2.8** The Backend subsystem shall provide API endpoints to extract and format class details and assignments from Brightspace.
- **5.3.2.9** The Backend subsystem shall provide API endpoints to extract and format Brightspace notifications.
- **5.3.2.10** The Backend subsystem shall monitor Brightspace API token expiration and utilize refresh tokens to maintain access without requiring continuous user intervention.

Account Management

- **5.3.2.11** The Backend subsystem shall provide API endpoints to create, edit, and delete user accounts.

Managing the usage of PDFs and URLs/Scraping

- **5.3.2.12** The Backend subsystem shall provide API endpoints for users to upload PDFs and URLs for processing.
- **5.3.2.13** The Backend subsystem shall extract and filter text content from uploaded PDFs.
- **5.3.2.14** The Backend subsystem shall scrape provided URLs and extract relevant HTML content.
- **5.3.2.15** The Backend subsystem shall transmit filtered textual data from PDFs and URLs to the Llama 3 API to parse assignment details into structured JSON objects.
- **5.3.2.16** The Backend subsystem shall implement error handling for the Llama 3 API integration to gracefully manage rate limits, timeouts, or unparseable responses.
- **5.3.2.17** The Backend subsystem shall write proposed assignments to the database with a 'proposed' status flag, and update this flag to 'active' upon user confirmation.
- **5.3.2.18** The Backend subsystem shall enforce a maximum file size limit (e.g., 5MB) on user-uploaded PDFs and reject files exceeding this limit to prevent processing timeouts.

5.3.3 Database (Firebase) Subsystem

Authentication & Security

- **5.3.3.1** The Database subsystem shall utilize Firebase Authentication to manage user credentials.
- **5.3.3.2** The Database subsystem shall enforce row-level security rules ensuring users can only access and modify their own assignments and profile data.
- **5.3.3.3** The Database subsystem shall securely store encrypted Brightspace access tokens associated with user profile records.

Data Collections & Schema

- **5.3.3.4** The Database subsystem shall maintain a Users collection to store user profiles, settings, and integration states.
- **5.3.3.5** The Database subsystem shall maintain an Assignments collection containing title, due date, details, source, and completion status fields.
- **5.3.3.6** The Database subsystem shall store proposed AI-extracted assignments with a status flag prior to user finalization.
- **5.3.3.7** The Database subsystem shall maintain a Classes collection to cache user Brightspace enrollments.

File Storage (If applicable)

- **5.3.3.8** The Database subsystem shall utilize Cloud Storage to temporarily store user-uploaded PDFs prior to processing.
- **5.3.3.9** The Database subsystem shall automatically delete uploaded PDFs from Cloud Storage following text extraction.
- **5.3.3.10** The Database subsystem shall ensure all documents in the Assignments collection include a creationTimestamp and lastUpdatedTimestamp to facilitate data sorting and auditing.

5.4. Performance requirements by CSC:

- **5.4.1 Dashboard Loading**
 - The system shall display the user's dashboard within 5 seconds.
- **5.4.2 Task Creation**
 - The system shall display a confirmation of successful manual task creation within 3-5 seconds after the user submits it.
- **5.4.3 Task Deletion**
 - The system shall display a confirmation of successful manual task deletion within 3-5 seconds.
- **5.4.4 Task Editing**
 - The system shall display updated task information within 3 seconds after the user saves changes.
- **5.4.5 Review Page**
 - The system shall display all collected assignments on the review page within 10 seconds after completion of data processing.
- **5.4.6 Search**
 - The system shall return search results within 3 seconds of search request.
- **5.4.7 Filtering**
 - The system shall update the displayed task list within 3 seconds after the filter is selected.
- **5.4.8 Data Processing**
 - The system shall process retrieved assignment information within 10 seconds of receiving the data.
 - The system shall convert the retrieved assignment information into the standardized task format before displaying it to the user.
 - The system shall identify potential duplicates before displaying the assignment list to the user.
 - The system shall notify the user if external data source is unavailable.
- **5.4.9 Database performance - Firebase**
 - The system shall save an approved task to the database within 3 seconds of user approval.
 - The system shall retrieve a user's task list within 3 seconds of request.
- **5.4.10 Errors**

- The system shall provide an error message when an external API request fails.
- The system shall prevent duplicate assignments creation when the same information is retrieved multiple times.
- The system shall receive from a temporary server failure without permanently losing already stored user's data.

5.5 Project Environment Requirements

5.5.1 Development Environment Requirements

The table below shows the necessary environmental requirements/tools used to develop the Gather system.

Category	Requirement
Device	PC/Laptop
Operating system	Windows/MacOS/Linux
Programming language	JavaScript
Frontend	React.js
Backend	Node.js
Backend framework	Express.js
Database	Firebase
AI model	Llama
Version Control	Git
Code Editor	Visual Studio Code
Browser	Chrome/Edge/Firefox

5.5.2 Execution Environment Requirements

The table below will show what is necessary for users to have access to, in order to use the Gather effectively.

Category	Requirement
Device	Computer or mobile device
Operating system	Modern supported OS (Windows 10 or later, MacOS 12 or later, etc.)
Browser	Modern web browser
Internet	Internet connection
Account	Registered user account